BANORTE

GUIA DE TRABAJO

SonarQube

PARA USO EXCLUSIVO DE

BANORTE SEGUROS Y PENSIONES

ELABORADO POR:

ARQUITECTURA TI

BANORTE

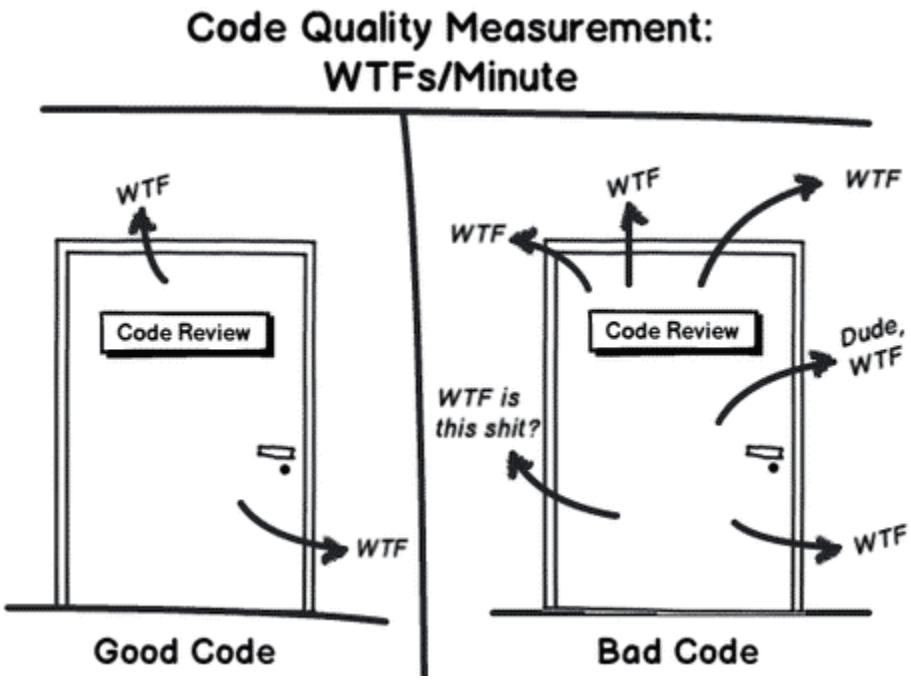
SonarQube

Contenido

1. ¿**Qué es la calidad del código**?
2. **Qué es SonarQube y para qué sirve**
3. **Por qué usar SonarQube**
4. **Análisis estático o dinámico del código fuente**
5. **Funcionalidades más relevantes de SonarQube**
6. **Estructura del sonar y CI**
7. **Guía de Instalación en Pasos de instalación**
8. **Reportes de código en SonarQube**
9. **Lenguajes soportados**
10. **Issues y Reglas**
11. **Quality Profile**
12. **Quality Gate**
13. **Vista del proyecto**
14. **Vista de issues**
15. **Vista de actividad**
16. **Complejidad ciclomática y cognitiva**

1. ¿Qué es la calidad del código?

Dicho por Patroklos Papapetrou su descripción de la calidad del código. Dice que "la calidad del código es un indicador de la rapidez con la que los desarrolladores pueden agregar valor comercial al sistema de software".



Características de calidad del software: ISO/IEC 9126
Para evaluar el software, es necesario seleccionar las características de calidad relevantes. ISO/IEC 9126 define un modelo de calidad que es aplicable a todo tipo de software. Define seis características de calidad del producto.

2. Qué es SonarQube y para qué sirve …

SonarQube es un software open source que permite hacer análisis estático del código y obtener métricas que permiten mejorar la calidad del código y descubrir evidencias de los problemas existentes en el código.

SonarQube calcula muchas métricas asociadas al código, y las métricas de deuda técnica están basadas en SQALE (SQALE es un plugin de SonarQube) , que podríamos definir como un algoritmo que dice sí las expectativas de calidad que se fijaron durante el ciclo de vida del desarrollo se han cumplido y/o en cuanto tiempo se va a demorar en alcanzarlas.

Sería buena praxis utilizar esta herramienta desde el inicio de los proyectos ya que permite conocer la evolución, analizando día a día lo que va sucediendo. Utilizar esta herramienta nos permitirá adaptar el workflow de trabajo para escribir mejor código. Esta nueva práctica va a evitar que se genere una deuda técnica muy grande y nos va a permitir mejorar la calidad del proyecto de forma natural.

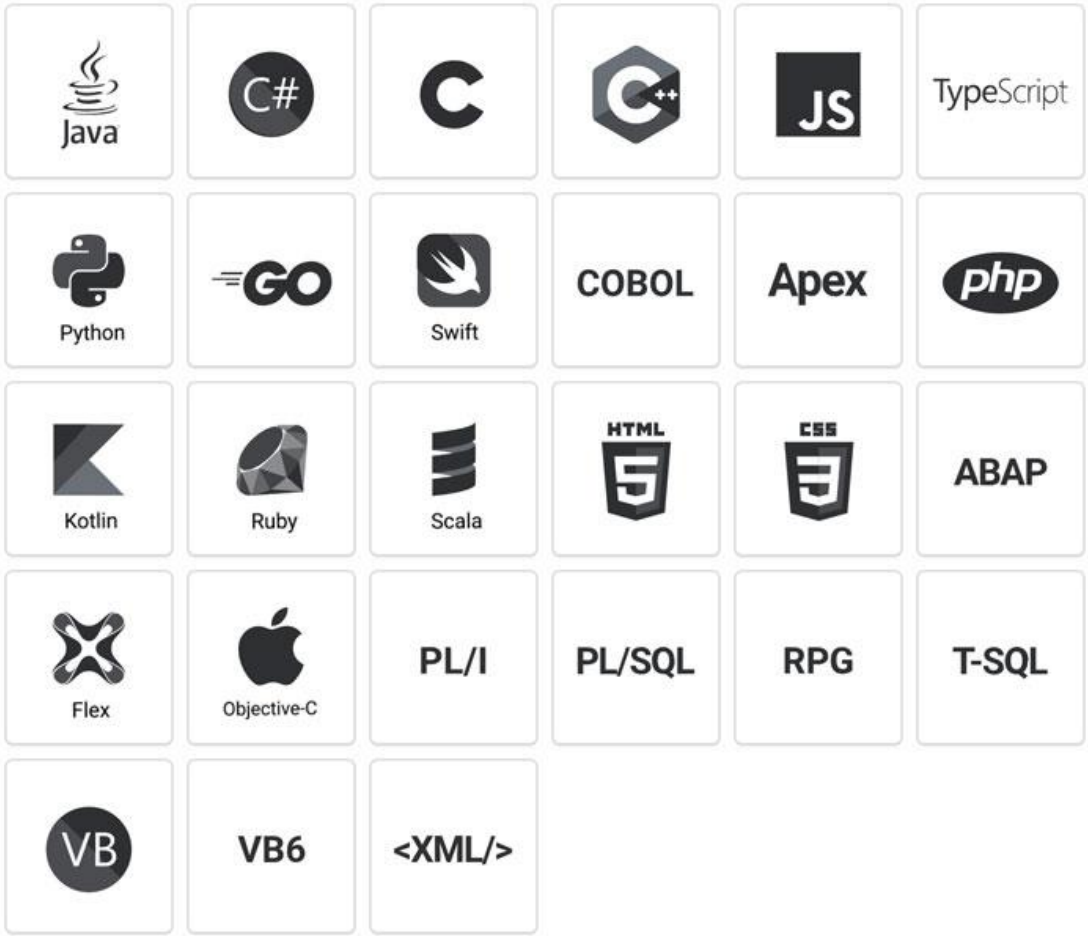
Entre las métricas tenemos el **código duplicado, cobertura, code smells, bugs, vulnerabilidades, densidad de evidencias de seguridad** y entre otros.

SonarQube está dividido en dos herramientas: el servidor donde se encuentra el motor de computación que realiza el análisis, almacena los resultados y es donde se puede definir los proyectos, parámetros de calidad, resultados análisis, quien se encargará de cada tarea y por el otro lado tenemos SonarQube&trade Scanner que es el analizador.

SonarQube no está limitado a un único lenguajes, se pueden hacer análisis de diferentes códigos y repositorios.

Análisis estático o dinámico del código fuente
Cuando se habla de análisis estático de código se entiende el proceso de evaluación del software sin ejecutarlo, en manera de poder obtener errores y métricas que hablen del código lo más pronto posible.

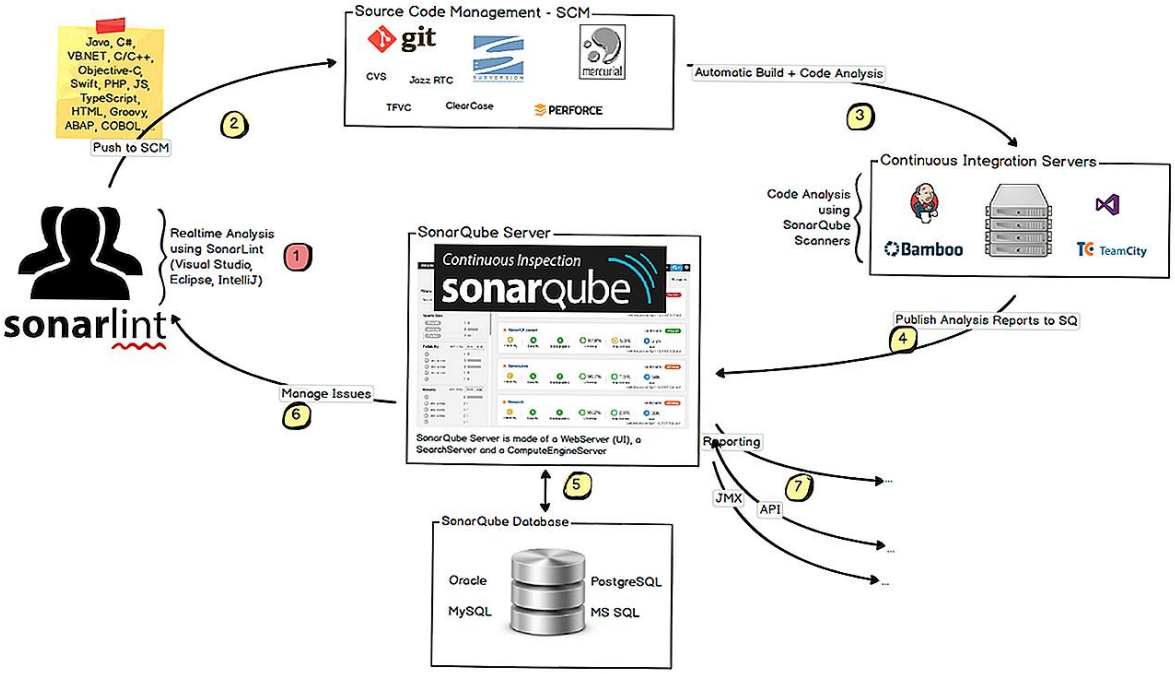
En cambio, para el análisis dinámico se tiene que ejecutar el software para ver el comportamiento y los tiempos. El problema en este caso es que se necesita más casos de prueba para poder definir que nuestro código está funcionando correctamente.



3. Por qué usar SonarQube:

En GFB utilizamos ambas son herramientas de análisis de código estático Fortify y SonarQube. Fortify se inclina más hacia la seguridad, ya que brinda información sobre las vulnerabilidades incluidas en OWASP, SANS, etc. Sonarqube también muestra esta información.

Fortify esencialmente clasifica los problemas de calidad del código en términos de su impacto de seguridad en la solución. Sonarqube es más una herramienta de análisis de código estático que también le brinda "olores de código" similares, Sonarqube también enumera las vulnerabilidades como parte de su análisis.



4. Análisis estático o dinámico de código fuente

Cuando se habla de análisis estático de código se entiende el proceso de evaluación del software sin ejecutarlo, en manera de poder obtener errores y métricas que hablen del código lo más pronto posible.

En cambio, para el análisis dinámico se tiene que ejecutar el software para ver el comportamiento y los tiempos. El problema en este caso es que se necesita más casos de prueba para poder definir que nuestro código está funcionando correctamente.

5. Funcionalidades más relevantes de SonarQube

Vulnerabilidades: SonarQube&trade posee un gran conjunto de reglas para detectar bugs, code smells y vulnerabilidades y así conocer el nivel de seguridad y calidad del proyecto de manera detallada.

Código duplicado: la herramienta detecta el código duplicado y nos permite tomar decisiones para refactorizar o desacoplar componentes.

Buenas prácticas de programación: permite saber si se incumplen los estándares y detecta las malas prácticas (code smells).

Cobertura: monitoriza sí la cobertura de las pruebas es suficiente para los estándares de calidad y si es necesario aumentar la inversión en los tests.

Perfiles de calidad: por cada lenguaje hay un perfil de calidad por defecto, pero también hay la posibilidad de crear un perfil personalizado. Se trata de un pack de reglas que se aplican durante el análisis estático del código.

SonarQube&trade es gratuita y open source - que con poco esfuerzo permite tener controlado el nivel de calidad. Y si estás cargo de muchos proyectos software te dará la posibilidad de tener todo bajo control.

6. Estructura del sonar y CI

La estructura se divide en 2 capas, sonarqube quien desarrolla los escaneos de código y sonarscanner quien aloja nuestro proyecto y administra los resultados .

La estructura de la carpeta está organizada para contener los binarios, las configuraciones, las extensiones, los registros y el área web del tablero de instrumentos.

Navega a la carpeta conf. La carpeta conf consta de dos archivos de configuración, wrapper.conf y sonar.properties. Wrapper.conf contiene configuraciones JVM para el servidor SonarQube, como la memoria limita las rutas JVM.

El sonar.properties contiene la configuración del servidor de sonarQube. Por defecto, el servidor SonarQube hace uso de una base de datos H2 incorporada.

Esta configuración no se recomienda para un entorno de producción.

El archivo contiene configuración para diferentes tipos de bases de datos y el tipo requerido de configuración de la base de datos podría habilitarse según sea necesario.

7. Guía de Instalación en Pasos de instalación

1) Ya teniendo Docker correctamente instalado, ejecutamos dentro de power shell: docker run -d --name sonarqube -p 9000:9000 -p 9092:9092.
De esta manera se instala nuestro contenedor de SonarQube .

2) Inicio SonarQube en el contenedor de docker con: "docker container start sonarqube"
(Puedes iniciarlo manual mente desde Docker Desktop en la opción de Contenedores -> SonarQube).

3) Una vez instalado y activo (tarda un poco la primera vez) entramos a la url : http://localhost:9000
Para ver nuestra plataforma de SonarQube en nuestro navegador .

4) Cambiamos nuestra contraseña de la que nos da por defecto admin/admin por la que elijamos ejemplo "admin/12345".
.
5) Dentro de sonar nos dirigimos al botón de nuevo proyecto

6) Elegimos crear un proyecto nuevo de forma manual.

7) Elegimos un nombre y una "key" (llave que por defecto tiende a ser el nombre de nuestro proyecto, pero puedes editarlo).

8) Elegimos la opción "locally" o "local" para escanear dentro de nuestro equipo , más adelante veremos cómo escanear código a través de repositorios.

9) Creamos una cadena de login (es un token de identificación que permite a sonar-scanner transferir datos a nuestro servidor SonarQube).

10) Seleccionamos la opción de lenguaje que corresponda.

11) Seleccionamos el sistema operativo (en el ejemplo usamos Windows con una imagen de Docker desktop , dependiendo de la opción que seleccionemos nos dará una línea de comando para ejecutar

dentro de nuestra carpeta donde esta alojada nuestro proyecto a escanear, igual mente nos da una para Linux , Windows y macOS)

12) Descargamos la carpeta sonar-scanner del sitio oficial y Establecemos las "variables de entorno de esta cuenta" dentro de nuestras variables de entorno dentro de Path
(original mente a mi no me dio problemas sin este paso, pero facilita que no tengamos que ubicar nuestro "sonar-scanner.bat" en el caso de windows a cada momento, yo recomiendo ubicarse en la carpeta donde queremos trabajar para siempre tener en cuenta donde estamos y que estamos escaneando)

13) Creamos los directorios donde alojaremos nuestro proyecto, esto se hiso con docker volume:

```
docker volume create sonarEscan

docker volume create libSonar
```

14) Una vez que localicemos nuestro directorio codeEscan colocamos dentro nuestro directorio (carpeta) del proyecto y creamos dentro un documento de nombre "sonar-project.properties"

15) Dentro del documento "sonar-project.properties" escribimos :

```
    sonar.projectKey="el nombre del proyecto, este lo vimos en el paso 7"

    sonar.host.url=http://localhost:9000
```

(nota: en esa ocasión solo agregue dentro del documento sonar-project.properties la instrucción sonar.projectKey , porque ya está descrita en nuestra cadena de escaneo)

16) Ejecutamos el escaneo con el siguiente código, que es en resumen otra imagen de docker que es el cliente de escaneos

```
    docker run \

        --rm \

        -e SONAR_HOST_URL="http://${SONAR_HOST}" \

        -e SONAR_LOGIN="${SONAR_TOKEN}" \

        -v "${REPO}:/usr/src" \

        -v $CONFIG_FILE:/opt/sonar-scanner/conf/sonar-scanner.properties \

        -v `pwd`/cacerts:/opt/java/openjdk/lib/security/cacerts \

        sonarsource/sonar-scanner-cli
```

Con la información de ejemplo

```
    docker run \

        --rm \

        -e SONAR_HOST_URL="http://localhost:9000" \

        -e SONAR_LOGIN="{el token que generamos en el paso 9}" \

        -v "sonarEscan:/usr/src" \


<---Nota : En lugar de Docker volume pueden ser directorio donde este el
código (C:/…..) en vez de solo "sonarEscan" así me funciona perfectamente pero
me a funcionado ignorar esta línea y agregar la ejecución directa de nuestro
sonar-scanner.bat directamente en el caso de windows ---<

        -v $CONFIG_FILE:/opt/sonar-scanner/conf/sonar-scanner.properties \ <--
yo no uso este --<

        -v `pwd`/cacerts:/opt/java/openjdk/lib/security/cacerts \ <-- yo no
uso este --<

        sonarsource/sonar-scanner-cli
```

17) regresamos a localhost:9000 y actualizamos para ver el resultado

Comentarios:
1 –
Descubrimos que para código Java con Maven hay que agregar esto:
" sonar.java.binaries.
Utilice la ruta a la carpeta que contiene los archivos de clases como valor. Agregue esta propiedad en el archivo sonar-project.properties.
Ejemplo, las clases están en el directorio 'target/classes', por lo que quedaría:
sonar.java.binaries=target/classes
Si no tiene, se excluyen agregando esta línea
sonar.exclusions=*.java "

2 – Encontré un problema en la instalación de sonar en Windows sin Docker.
Esta causa un error al no encontrar el jdk y lo solucione dentro de la carpeta de sonarqube que descargamos -> conf, dentro de wrapper:
Agregamos en la línea wrapper.java.command= la ruta de nuestro jdk:
wrapper.java.command=C:/Program Files/Java/jdk-17.0.1/bin/java

8. Reportes de código en SonarQube

Código duplicado
Este término se utiliza cuando hablamos de un código fuente que aparece más de una vez, ya sea dentro de uno o diferentes programas, de propiedad o mantenido, por la misma entidad.
La duplicación de código es generalmente considerada una señal de estilo de programación pobre, ya que un buen desarrollo está más asociado a la reutilización del mismo.
La principal desventaja es el mantenimiento del código, lo cual se convierte en una tarea mucho más costosa.

Código muerto
Es el código que se encuentra en nuestra aplicación, pero nunca es utilizado. Normalmente aparece después de hacer refactor en nuestro código.

Estándares de codificación
Se refiere a convenciones para escribir código fuente, las cuales frecuentemente son dependientes del lenguaje de programación.
Las convenciones más comunes hacen referencia a: nombres de variables, indentaciones, espaciado...

Bugs
Un bug es un error o un defecto en el software que hace que un programa funcione de forma incorrecta.

Complejidad ciclomática *
Es una métrica de calidad software basada en el cálculo del número de caminos independientes que tiene nuestro código.
Cuanto más compleja sea la lógica de un código, más difícil será de entender, mantener y probar.

Comentarios
Los comentarios son añadidos usualmente con el propósito de hacer el código fuente más fácil de entender con vistas a su mantenimiento o reutilización.
Una mala o escasa documentación puede convertir el mantenimiento del código en una tarea muy costosa.
Al igual que ocurre con los tests, un gran porcentaje de comentarios no asegura calidad. Los comentarios deben ser buenos y aclaratorios explicando interfaces públicas, pero no lógica ni diseño.

Tests unitarios y de integración
Son una forma de comprobar el correcto funcionamiento de una unidad de código y de la integración de mismos.
Facilitan los cambios en la aplicación, ya que las pruebas nos aseguran que los nuevos cambios no han introducido errores en partes de código desarrolladas anteriormente y que están cubiertas por estos.
Por otro lado, documentan el código, siendo un libro abierto sobre el funcionamiento del código.

Cobertura de código
La cobertura de código es una medida que nos indica el porcentaje de código validado por los tests. Generalmente con una mayor cobertura aseguramos que no se introducen errores en una mayor parte del código, pero esto dependerá de la funcionalidad real que cubran los tests.

Análisis Estático
Se denomina análisis estático del código al proceso de evaluar el software sin ejecutarlo. Pese a esto, es importante aclarar que si queremos comprobar la cobertura de los tests, estos sí deben ser ejecutados.

9. Lenguajes soportados

Podemos analizar más de 20 lenguajes de programación. Cada analizador proporciona numerosas reglas para detectar problemas de calidad generales y específicos del lenguaje.

- C/C++
- Javascript
- C#
- Java
- COBOL
- TypeScript
- PL/SQL
- PL/I
- PHP
- ABAP
- T-SQL
- VB.NET
- VB6
- Python
- RPG
- Flex
- Objective-C
- Swift
- Web
- XML

Checkstyle, PMD y Findbugs
Sonarqube usa diversas herramientas de análisis estático como Checkstyle, PMD o FindBugs para obtener métricas que pueden ayudar a mejorar la calidad, a través de las reglas que estas definen.

	Checkstyle	PMD	Findbugs
Propósito	Verificar el cumplimiento de las reglas de codificación.	Identificación de problemas potenciales.	Encontrar errores.
Tipos de verificación	- Convenciones de nombres - Encabezados, importaciones - Espacios en blanco, formateo - Comentarios de javadoc - Buenas prácticas, convenciones de códigos - Parámetros del método - Complejidad ciclomática - Cuialquier expresión regular	- Posibles errores - Código muerto - Código duplicado - Complejidad ciclomática - Expresiones sobrecomplicadas (legibilidad)	- Posibles errores - Defectos de diseño - Malas prácticas - Correción multiproceso (correcto manejo sincronía) - Vulnerabilidades de código

10. Issues y Reglas

Cuando un componente no cumple con una regla de codificación de las definidas por Checkstyle, PMD, Findbugs..., se registra una issue.
Hay 3 tipos de issues:

- Bug: un punto de fallo real o potencial en su software.
- Vulnerability: un agujero de seguridad que puede usarse para atacar su software.
- Code Smells: un problema relacionado con la mantenibilidad en el código. Los futuros desarrolladores tendrán más dificultades de las que deberían para realizar cambios en el código y esto puede provocar que introduzcan errores adicionales a medida que realizan cambios.

11. Quality Profile

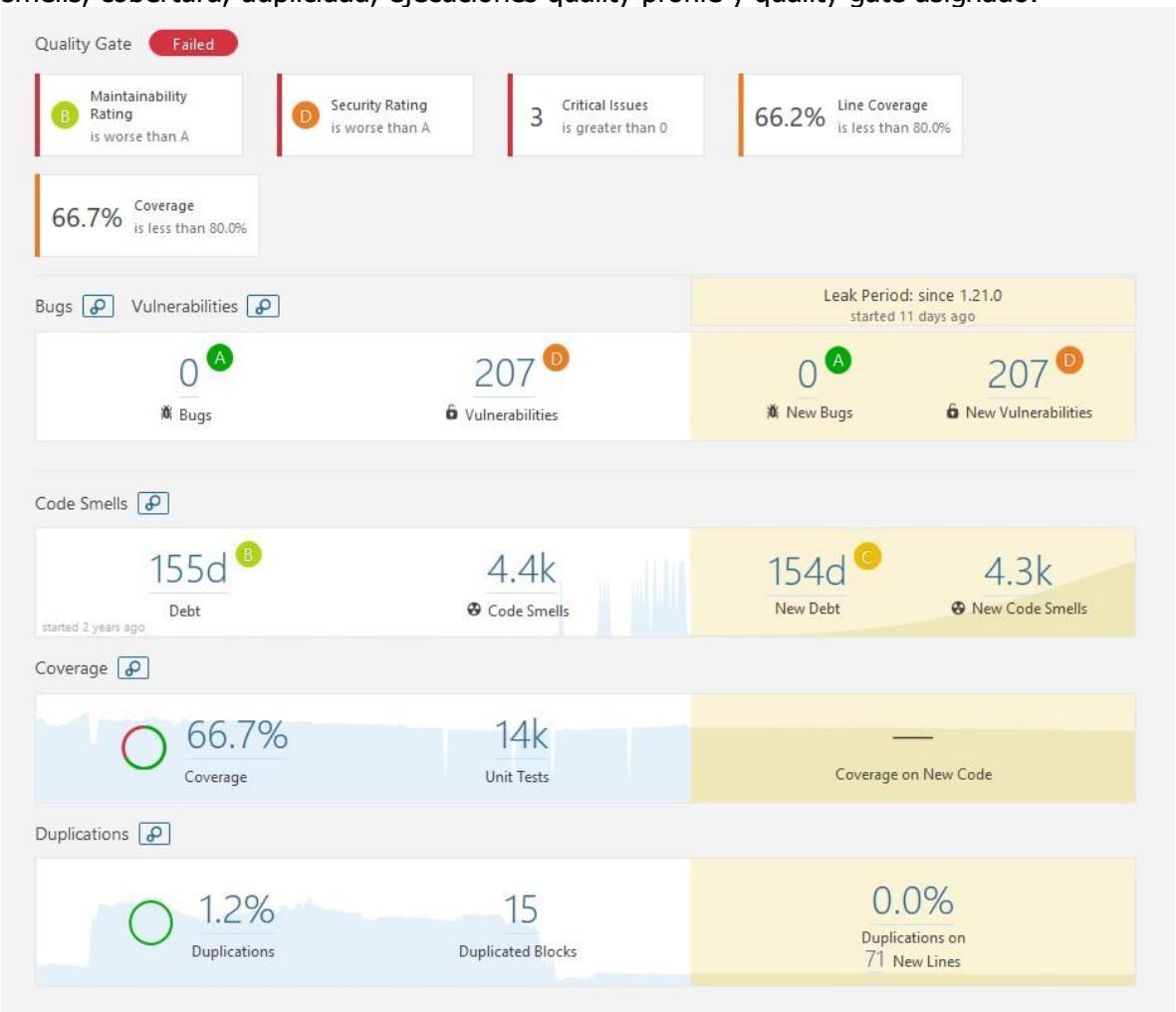
Son colecciones de reglas que se revisan durante un análisis. Para cada tecnología hay un Quality Profile predeterminado, pero podemos tener varios para un mismo lenguaje.
En caso de que un proyecto analizado no tenga asignado explícitamente uno, se analizará con el perfil configurado como predeterminado para dicho lenguaje.

12. Quality Gate

Es el conjunto de condiciones que el proyecto debe cumplir antes de que pueda ser lanzado a producción. Al igual que ocurre con los quality profiles, es posible establecer un Quality Gate predeterminado.
Nota: siempre que se pueda está bien tener un mismo quality gate y quality profile para todos los proyectos de la empresa. Esto nos garantiza un estándar de calidad dentro de la compañía y también hace más liviano la movilidad de los desarrolladores entre proyectos.

13. Vista del proyecto

Cada proyecto dispondrá de una pantalla con el resumen del mismo, la cual se actualizará después de cada análisis. Desde esta vista podemos información referente a: bugs, vulnerabilidades, code smells, cobertura, duplicidad, ejecuciones quality profile y quality gate asignado.

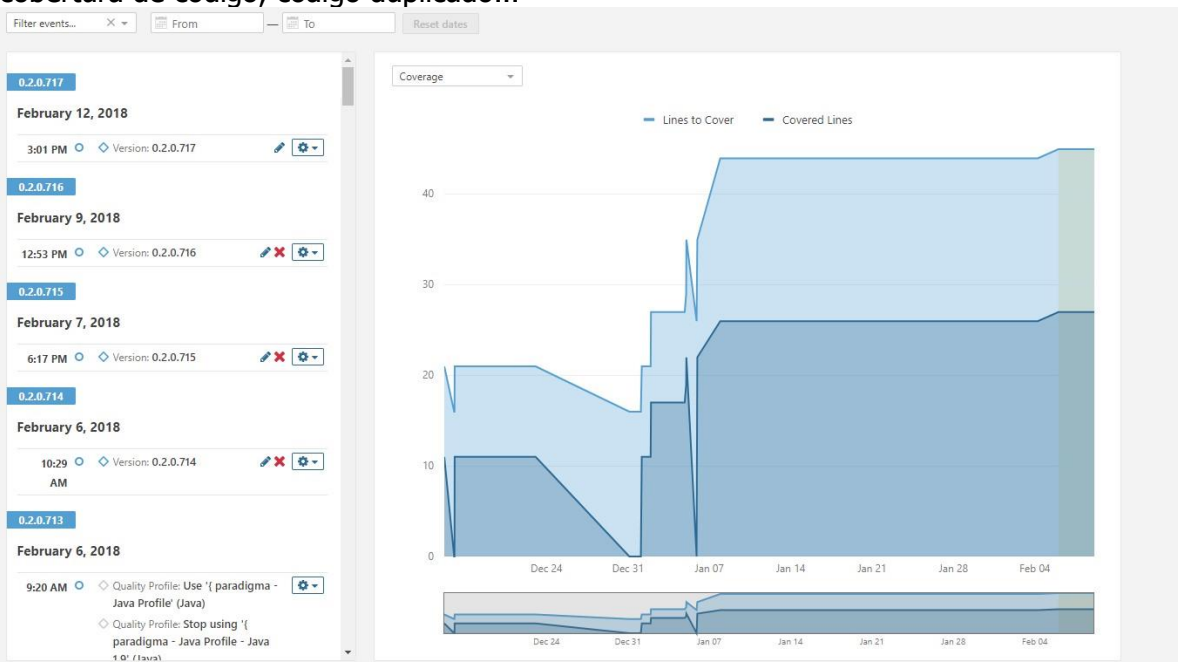


14. Vista de issues

Desde esta vista podemos gestionar y obtener más detalles de las issues pendientes. Pulsando en cada una de ellas podemos ver la línea exacta donde que genera la issue, una breve descripción de la misma e incluso se nos ofrecerá una posible solución para solventarla.

15. Vista de actividad

Desde aquí podemos ver un historial de ejecuciones y sus resultados. Este apartado es muy útil para sacar métricas y poder ir observando cómo van mejorando ciertos aspectos como: número de issues, cobertura de código, código duplicado...



Una vez conocidas las principales características que miden la calidad de código, y después de haber indagado en Sonarqube, solo os queda seguir investigando sobre esta herramienta e introducirla como una nueva fase en el flujo de integración continua de vuestro proyecto.
Como comentaba al inicio del post, cada vez son más las personas y clientes concienciados de la importancia de la calidad del código, pero hay que seguir luchando para que sean muchos más y desde Paradigma os animamos a ello.

16. Complejidad ciclomática y cognitiva

Hago especial nota sobre las complejidades de código al ser de referencia al interpretar uno de los errores mas comunes para reportar código

Es la complejidad ciclomática calculada en función del número de rutas a través del código.

Cada vez que el flujo de control de una función se divide, el contador de complejidad se incrementa en uno. Cada función tiene una complejidad mínima de 1. Este cálculo varía ligeramente según el idioma porque las palabras clave y las funcionalidades lo hacen.

Mientras que la complejidad cognitiva describe una nueva métrica que rompe con el uso de matemáticas modelos para evaluar el código con el fin de remediar las deficiencias de Cyclomatic Complexity y
producir una medición que refleje con mayor precisión la dificultad relativa de comprensión,
y, por lo tanto, de mantener métodos, clases y aplicaciones.

Más Información Departamento de Arquitectura TI Autores de publicación:

Claudio Alonso Vargas claudio.vargas.organista@banorte.com
Mario Ignacio Ledesma Merino mario.ledesma.merino@banorte.com
Arturo Hernández Ambriz arturo.hernandez.ambriz@banorte.com